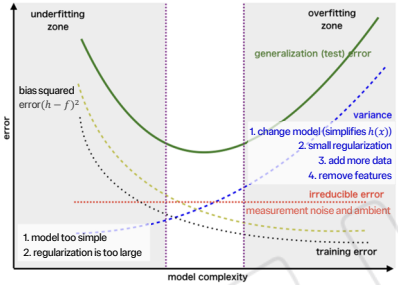
1.4.1
digital thread trace of product lifecycle
digital twin real-time system virtual replica

1.5
supervised regression, classification, prediction, ...
unsupervised clustering, dimensionality reduction, ...
semi-supervised clustering 聚类 → labelling

2 terminology
features - input, $\boldsymbol{x}^{(i)} = (1, x_1{}^{(i)}, x_2{}^{(i)}, \dots, x_N{}^{(i)})$ * $\mathbf{x} = \boldsymbol{x}^{\mathrm{T}}$
* e.g. $\boldsymbol{x}^{(i)} = (1, x^{(i)}, x^{(i)2}, \dots, x^{(i)N})$
label/target - i-th data point of the feature, $y^{(i)}$
training set - $\mathbf{y} = (y^{(1)}, y^{(2)}, \dots, y^{(m)})^{\mathrm{T}}$ *classes in classification
bias - θ_0
weights - θ_i
parameters - $\boldsymbol{\theta}_{(n)} = (\theta_0, \theta_1, \dots, \theta_N)_{(n)}$ * $\boldsymbol{\theta}_{(n)} = \boldsymbol{\theta}_{(n)}{}^{\mathrm{T}}$
hypothesis function - $h(\boldsymbol{x}^{(i)}) = \boldsymbol{\theta} \cdot \boldsymbol{x}^{(i)} = \theta_0 + \theta_1 x_1{}^{(i)} + \cdots$
loss function - $J_{(n)}(\boldsymbol{\theta})$
learning rate - α
regularization parameter - λ
epoch - how many times let model learn from data
convex - local = global minimum, $f(y) \geq f(x) + \nabla f(x) \cdot (y - x)$
* e.g. $\exp(sx), s \in \mathbb{R}, n = 2,4, \dots, -\ln(x), x > 0, \frac{1}{x^r}, x, r > 0$

loss function convex for linear $h(x)$
mean-squared error (mse) - $\frac{1}{2m}\sum_{i=1}^{m}(h(x_k{}^{(i)}) - y^{(i)})^2$
mean absolute error (mae) - $\frac{1}{2m}\sum_{i=1}^{m}|h(x_k{}^{(i)}) - y^{(i)}|$
Huber loss - mse for small and mae for large errors * prefer in **6**

1.3 - 1.5
multi-variable linear regression
1. choose starting point $\boldsymbol{\theta}_{(0)} = (\theta_0, \theta_1, \dots, \theta_N)_{(0)}{}^{\mathrm{T}}$
2. calculate gradient $\nabla_{\boldsymbol{\theta}} \boldsymbol{J}_{(0)} = (\frac{\partial J}{\partial \theta_0}, \frac{\partial J}{\partial \theta_1}, \dots, \frac{\partial J}{\partial \theta_N})_{(0)}{}^{\mathrm{T}}$
* **vectorization** $\nabla_{\boldsymbol{\theta}} \boldsymbol{J} = \frac{1}{m}\mathbf{X}^{\mathrm{T}}(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}), \mathbf{X} = (\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \dots, \boldsymbol{x}^{(m)})^{\mathrm{T}}$
3. update $\boldsymbol{\theta}_{(1)} = \boldsymbol{\theta}_{(0)} - \alpha \nabla_{\boldsymbol{\theta}} \boldsymbol{J}_{(0)}$
4. repeats until converge to tolerance or reach maximum iter.



1.6
regularization reduce test error * strictly convex

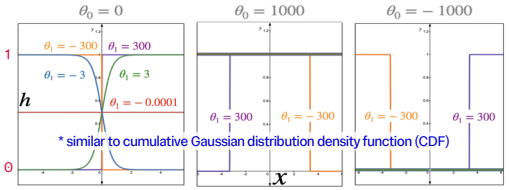
$$J(\boldsymbol{\theta}) = \frac{1}{2m}\sum_{i=1}^{m}(h(x_k{}^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{N}\theta_j{}^2$$

1.7.2
feature scaling magnitude of x_1 much larger than x_2
$\hat{x}_j = \frac{x_j - \mu_j}{\sigma}$ * also good for logistic regression, SVM, PCA, k-NN, k-means, DBSCAN,...

1.8
batch gradient descent use whole set per iter
* prone to overfitting and have memory requirement
stochastic 随机的 ~ use a random data point per iter
mini-batch ~ use a small batch of size m_b per iter
* easily parallelized in a computer software

3 1.2
logistic regression uses probability to do binary classification
sigmoid uncertainty of the feature vector belongs to a set (class)

$$\sigma(x) = h(y = 1|x) = \frac{1}{1 + e^{-\theta \cdot x}} = \frac{1}{1 + e^{-\theta_0 - \theta_1 x_1 - \cdots - \theta_N x_N}} \in (0, 1)$$
$$* \frac{\partial}{\partial x}\sigma(x) = \sigma(x)(1 - \sigma(x))$$

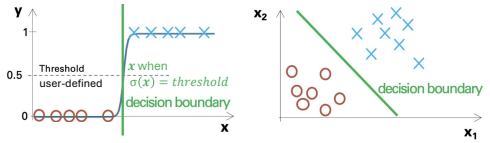


1.3
(average) cross-entropy loss function convex for $h(x) = \sigma(x)$

$$J(\boldsymbol{\theta}) = J_{y^{(i)}=1} + J_{y^{(i)}=0} = -\frac{1}{m}\sum_{i=1}^{m}\{y^{(i)}ln[h(x^{(i)})] + (1 - y^{(i)})ln[1 - h(x^{(i)})]\}$$

$$* \nabla_{\boldsymbol{\theta}} \boldsymbol{J} = \frac{1}{m}\mathbf{X}^{\mathrm{T}}(\sigma(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y}), \text{need to apply } \sigma \text{ to } \mathbf{X}\boldsymbol{\theta} \text{ elementwise}$$

1.4
logistic regularization same as regularization * strictly convex

1.5


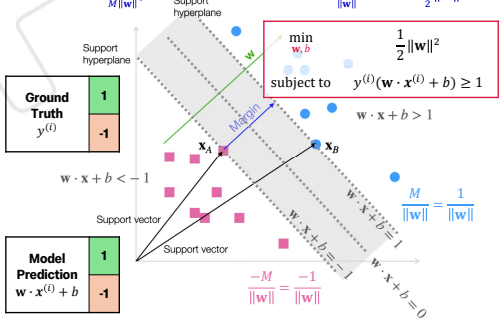
1.6
multiclass problems one-vs-all
n. of logistic regression classifiers needed = n. class

1.7
confusion matrix

		Ground Truth $y^{(i)}$ 1	Ground Truth $y^{(i)}$ 0	
Model Prediction $\sigma(x) = h(x)$	1	True Positive	False Positive	Precision $\frac{TP}{TP+FP}$
	0	False Negative	True Negative	
Accuracy $\frac{TP+TN}{TP+FP+FN+TN}$		Recall $\frac{TP}{TP+FN}$	F-score $2\frac{Precision \times Recall}{Precision + Recall}$	*ideally = 1

2.3
hyperplane $\mathbf{w} \cdot \boldsymbol{x} + b = \text{orthogonal} + \text{shift} = \mathbf{w} \cdot (\boldsymbol{x} - \boldsymbol{x_0}) = 0$
distance of $\boldsymbol{x_B}$ from hyperplane is $M = \frac{f(x_B)}{\|\mathbf{w}\|} = \frac{\mathbf{w} \cdot x_B + b}{\|\mathbf{w}\|} = \frac{\mathbf{w} \cdot (x_\perp + M\mathbf{w}/\|\mathbf{w}\|) + b}{\|\mathbf{w}\|}$

2.4
support vector machine (SVM) when n.features > n.examples
classification - find maximum-margin separation hyperplane, maximize $\frac{2}{\|\mathbf{w}\|}$
* normalize $\mathbf{w} \rightarrow \frac{\mathbf{w}}{M\|\mathbf{w}\|}$, maximize $\frac{2}{\|\mathbf{w}\|}$ = minimize $\frac{1}{2}\|\mathbf{w}\|^2$



regression (SVR) - finds a function that is as flat as possible, minimize slope $\|\mathbf{w}\|$

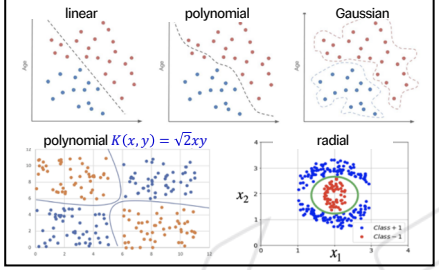
2.4.2
regularization and soft margins * strictly convex

$$\min_{\mathbf{w}, b, \xi} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i^m \xi_i \quad * C \text{ is box contstraint}$$
$$\text{subject to} \quad y^{(i)}(\mathbf{w} \cdot \boldsymbol{x}^{(i)} + b) \geq 1 - \xi_i, \text{for } \xi_i \geq 0$$

* small C – soft (wide) margin, C → ∞ - hard margin

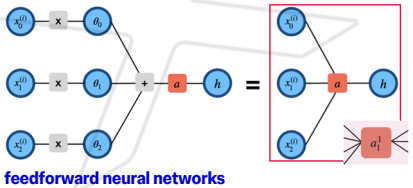
2.5
nonlinear separable classes feature mapping
enriching the data - $f(\boldsymbol{x}^{(i)}) = \mathbf{w} \cdot \boldsymbol{\Phi}(\boldsymbol{x}^{(i)}) + b$
* e.g. $\boldsymbol{\Phi}(x_1{}^{(i)}, x_2{}^{(i)}) = (x_1{}^{(i)}, x_2{}^{(i)}, x_1{}^{(i)2} + x_2{}^{(i)2})$
* computationally expensive by adding di explicitly

kernal trick - $f(\boldsymbol{x}^{(i)}) = \sum_{j=1}^{m}\beta_j \boldsymbol{\Phi}(\boldsymbol{x}^{(j)}) \cdot \boldsymbol{\Phi}(\boldsymbol{x}^{(i)}) + b = \sum_{j=1}^{m}\beta_j \boldsymbol{K}(\boldsymbol{x}^{(i)} \cdot \boldsymbol{x}^{(i)}) + b$
* e.g. polynomial, $\boldsymbol{K}(\boldsymbol{y} \cdot \boldsymbol{x}) = (\boldsymbol{y} \cdot \boldsymbol{x} + 1)^2$
radial basis function, $\boldsymbol{K}(\boldsymbol{y} \cdot \boldsymbol{x}) = \exp(-\gamma\|\boldsymbol{y} - \boldsymbol{x}\|^2)$
* enables higher-dimensional feature space without cost

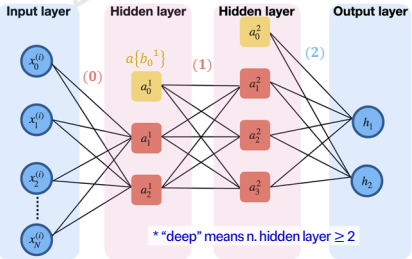


4 1.3 – 1.5
nonlinear function linear combination of linear function
shift - rotate – add up – deform – repeat

one neuron



feedforward neural networks



input layer - $\boldsymbol{x}^{(i)} = (x_0{}^{(i)}, x_1{}^{(i)}, x_2{}^{(i)}, \dots, x_N{}^{(i)})^{\mathrm{T}}$ * $x_0{}^{(i)} = 1$
1st hidden layer –

$$\boldsymbol{\theta}^{(0)} = \begin{pmatrix} \theta_{0,1}{}^{(0)} & \theta_{0,2}{}^{(0)} \\ \theta_{1,1}{}^{(0)} & \theta_{1,2}{}^{(0)} \\ \vdots & \vdots \\ \theta_{N,1}{}^{(0)} & \theta_{N,2}{}^{(0)} \end{pmatrix}, \boldsymbol{z}^1 = \begin{pmatrix} b_0{}^1 \\ \boldsymbol{\theta}^{(0)\mathrm{T}}\boldsymbol{x} \end{pmatrix}, \boldsymbol{a}^1 = a\{\boldsymbol{z}^1\} = \begin{pmatrix} \boldsymbol{a}_0{}^1 \\ \boldsymbol{a}_1{}^1 \\ \boldsymbol{a}_2{}^1 \end{pmatrix}$$

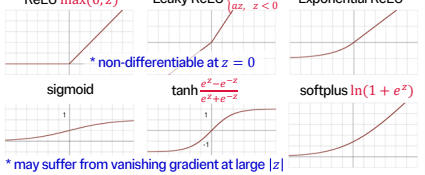
2nd hidden layer –

$$\boldsymbol{\theta}^{(1)} = \begin{pmatrix} \theta_{0,1}{}^{(1)} & \theta_{0,2}{}^{(1)} & \theta_{0,3}{}^{(1)} \\ \theta_{1,1}{}^{(1)} & \theta_{1,2}{}^{(1)} & \theta_{1,3}{}^{(1)} \\ \theta_{2,1}{}^{(1)} & \theta_{2,2}{}^{(1)} & \theta_{2,3}{}^{(1)} \end{pmatrix}, \boldsymbol{z}^2 = \begin{pmatrix} b_0{}^2 \\ \boldsymbol{\theta}^{(1)\mathrm{T}}\boldsymbol{a}^1 \end{pmatrix}, \boldsymbol{a}^2 = a\{\boldsymbol{z}^2\}$$

output layer –

$$\boldsymbol{\theta}^{(2)} = \begin{pmatrix} \theta_{0,1}{}^{(2)} & \theta_{0,2}{}^{(2)} \\ \theta_{1,1}{}^{(2)} & \theta_{1,2}{}^{(2)} \\ \theta_{2,1}{}^{(2)} & \theta_{2,2}{}^{(2)} \\ \theta_{3,1}{}^{(2)} & \theta_{3,2}{}^{(2)} \end{pmatrix}, \boldsymbol{z}^3 = \begin{pmatrix} b_0{}^2 \\ \boldsymbol{\theta}^{(2)\mathrm{T}}\boldsymbol{a}^2 \end{pmatrix}, \boldsymbol{h} = \boldsymbol{a}^3 = a\{\boldsymbol{z}^3\}$$

1.8
activation function all a below are monotonic increasing



* may suffer from vanishing gradient at large |z|

1.8
output layer activation function varies with task
regression – linear units * $\boldsymbol{J}$ ='MeanSquaredError()'
binary classification $\{0, 1\}$ - sigmoid * $\boldsymbol{J}$ = 'BinaryCrossentropy()'
multi-class classification $\{0, 1, \dots, k\}$ – softmax
* $\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{i=1}^{k}\exp(z_i)}$ = probability of feature belongs class i

Label Encoding

Food Name	Categorical	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

'SparseCategoricalCrossentropy()'

One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

'CategoricalCrossentropy()'

1.6
number of parameters
n. of paramaters = $\sum_{i=0}^{L}(s_i + 1)\, s_{i+1}$ * for input layer, $s_0 = N$

1.7
forward propagation evaluate loss function
regression -

$$J(\boldsymbol{\theta}) = \frac{1}{2m}\sum_{i=1}^{m}\sum_{k=1}^{K}\left[h_k(\boldsymbol{x}^{(i)}) - y_k^{(i)}\right]^2 + \lambda\sum_{l=0}^{L}\sum_{a=0}^{s_L}\sum_{b=0}^{s_L}(\theta_{a,b}^{(l)})^2$$

classification -

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\{y_k^{(i)}ln[h_k(\boldsymbol{x}^{(i)})] + (1 - y_k^{(i)})ln[1 - h_k(\boldsymbol{x}^{(i)})]\} + \lambda\sum_{l=0}^{L}\sum_{a=0}^{s_L}\sum_{b=0}^{s_L}(\theta_{a,b}^{(l)})^2$$

* K = n. of output
* loss function may non-convex, very likely local ≠ global minimum

1.10
backpropagation evaluate gradient of loss function
chain rule -

$$\frac{dJ}{d\boldsymbol{\theta}^{(l)}} = \frac{dJ}{d\boldsymbol{h}}\frac{d\boldsymbol{h}}{d\boldsymbol{a}^{L+1}}\frac{d\boldsymbol{a}^{L+1}}{d\boldsymbol{\theta}^{(l)}} = \frac{dJ}{d\boldsymbol{h}}\frac{d\boldsymbol{h}}{d\boldsymbol{a}^{L+1}}\frac{d\boldsymbol{a}^{L+1}}{d\boldsymbol{z}^{L+1}}\frac{d\boldsymbol{z}^{L+1}}{d\boldsymbol{a}^{L}}\frac{d\boldsymbol{a}^{L}}{d\boldsymbol{z}^{L}}\cdots\frac{d\boldsymbol{z}^{l}}{d\boldsymbol{\theta}^{(l)}}$$
$$* \frac{d\boldsymbol{z}^{L+1}}{d\boldsymbol{a}^{l}} = \frac{d\boldsymbol{\theta}^{(l)}\boldsymbol{a}^{l}}{d\boldsymbol{a}^{l}} = \boldsymbol{\theta}^{(l)\mathrm{T}}, \frac{d\boldsymbol{a}^{l}}{d\boldsymbol{z}^{l}} = \frac{da(z^l)}{dz^l} = a'(z^l)$$

backpropagation -

$$\frac{d\boldsymbol{h}}{d\theta_{a,b}{}^{(l)}} = \frac{d\boldsymbol{h}}{d\boldsymbol{z}^{l+1}}\frac{d\boldsymbol{z}^{l+1}}{d\theta_{a,b}{}^{(l)}} = \boldsymbol{\delta}_b{}^{l+1}\boldsymbol{a}_a{}^{l}$$

*$\boldsymbol{\delta}$ is error at layer l, $\boldsymbol{\delta}_j{}^{l} = (\cdot)(\boldsymbol{z}_j^{l})$ in $\boldsymbol{\delta}^{l+1}\boldsymbol{\theta}^{(l)\mathrm{T}}a'(\boldsymbol{z}^{l})$
* involves only multiplication operations as forward store in memory

1.5
TensorFlow notation Python start from 0 instead of 1
input layer - $\boldsymbol{x} = (x_0, x_1, x_2, \dots, x_{N-1})$
l-th hidden layer –

$$\boldsymbol{\theta}^{(l)} = \begin{pmatrix} \theta_{0,0}{}^{(l)} & \cdots & \theta_{0,n_l-1}{}^{(l)} \\ \theta_{1,0}{}^{(l)} & \cdots & \theta_{1,n_l-1}{}^{(l)} \\ \vdots & \ddots & \vdots \\ \theta_{n_{l-1}-1,0}{}^{(l)} & \cdots & \theta_{n_{l-1}-1,n_l-1}{}^{(l)} \end{pmatrix} \quad * n_l = \text{n. of node in layer}$$

$$\boldsymbol{b}^{(l)} = (b_1{}^{(l)}, b_2{}^{(l)}, \dots, b_N{}^{(l)}), \boldsymbol{z}^l = \boldsymbol{a}^{l-1}\boldsymbol{\theta}^{(l)} + \boldsymbol{b}^{(l)}, \boldsymbol{a}^l = a\{\boldsymbol{z}^l\}$$

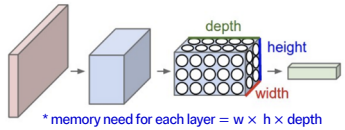
1.11
architecture design
fine tuning – finding good set of hyperparameters
n. of nodes – prefer same number of neurons in all hidden layers
n. of layers – n. of neurons exponentially related to n. of layers
* deep learning repeat composition functions (layers) to reduce n. of required neurons by a large factor, but often harder to train
* aways start with more layers and nodes than actually need

1.12
repeat
training initialize – 1.7 – 1.10 – gradient descent
shuffle 打乱 the data randomly * buffer size ≥ data size
split data in training (60-80%), validation (10-20%) and test (10-20%) * test set never use to train model

optimizer
standard gradient descent (sgd) – fixed learning rate α
* may get stuck in local minima
AdaGrad – adapative α, accumulate all past gradients
* learning rate decrease fast to small values
AdaDelta – adapative α based on moving window
RMSProp – adapative α using exponential moving avg.
Adam – RMSProp + momentum * fast and robust
* Nesterov momentum gradient descent, β typically 0.9

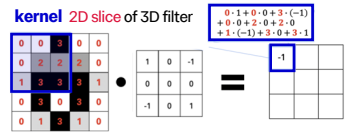
$$\boldsymbol{\theta}_{(n+1)} = \boldsymbol{\theta}_{(n)} - \alpha[(1 - \beta)\nabla_{\boldsymbol{\theta}}\boldsymbol{J}_{(n)} + \beta\nabla_{\boldsymbol{\theta}}\boldsymbol{J}_{(n-1)}]$$

5 1.3
CNNs convolutional neural networks
* fully-connected NN contain too many parameters for grid-like data, such as images
* sptial 空间的 dependencies are lost in **4**
CNN arrange neurons in 3D – width × height × depth



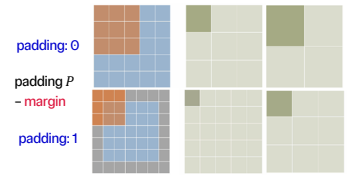
* memory need for each layer = w × h × depth

1.4
filter weight average of pixels by convolution
kernel 2D slice of 3D filter



input image raw pixel values * depth=3 for RGB

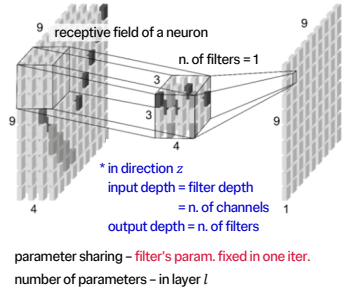
convolution layer set of learnable filters
stride S – step = resolution stride: 1 stride: 2



number of neurons – in direction $d = x, y$
* input length W_d, filter width F_d

$$\frac{W_d - F_d + 2P_d}{S_d} + 1$$

receptive field of the neuron – 3D region in one conv.
* spatially local along filter w & h, fully along input depth
depth K – n. of filters = n. of feature (activation) maps



parameter sharing – filter's param. fixed in one iter.
number of parameters – in layer l
$(F_x \times F_y \times \text{n. of channels} + 1) \times \text{n. filters}$

activation layer introduce nonlinearity, typically ReLU

* do not change size of the volume ($w \times h \times depth$)
* part of the convolution layer in theory, but explicit in TensorFlow

output after convolution (and activation) layer – feature map in layer l

$$z_{i,j,k} = a\left(\sum_{u=0}^{F_x-1}\sum_{v=0}^{F_y-1}\sum_{w=0}^{F_z-1} b_k + x_{(is_x+u),(js_y+v),(ks_z+w)} \cdot \theta_{u,v,w}\right)$$

pooling layer reduce the spatial size, no trainable param.

* max-pooling discarding 75% of act. layer with $F = 2,\ S = 2, P = 1$
* start with double the n. of filters after each pooling

flatten layer → fully-connected layer feedforward NN **4**

1.5

augment data regularization prevent overfitting

artificially increase the data size by generating realistic variations of the training data points - shift, rotate, risize, contrast, ...

dropout regularization prevent overfitting

dropout p% neurons at every training step except output layer

* dropout rate p typically set to $40 - 50\%$

coding TensorFlow

tf.keras.layers.Conv(n. filters, (filter volume), activation, input_shape)

* default setting gives strides (1, 1) and padding 'valid' → $P = 0$

6 **p3 – p13**

sequential data both data and order contain information

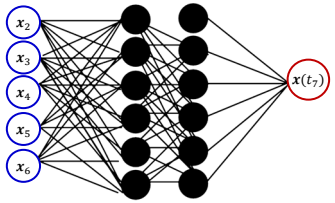
speech recognition, music generation, sentiment 观点 classification, DNA sequence analysis, machine translation, name recognition, video activity classification, time series, dynamical systems, sensor...

sigmoid uncertainty of the feature vector belongs to a set (class)

p14 – p26

sequence modelling with feedforward NN **4**

typically not good approach - can only retain memory for short past

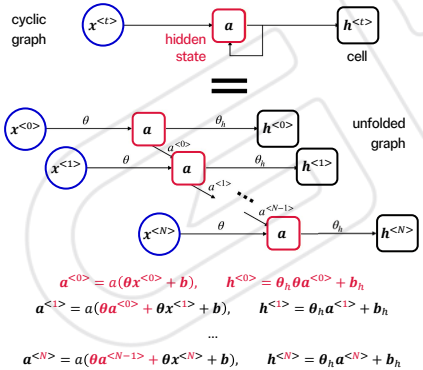


* do not learn correlation in time, interpolate instead
* has large parameters
* need inputs of fixed size

p27 – p42

recurrent cell *neuron in RNN

at time step t * also known as time stamp or time frame



$$a^{<0>} = a(\theta x^{<0>} + b), \quad h^{<0>} = \theta_h \theta a^{<0>} + b_h$$
$$a^{<1>} = a(\theta a^{<0>} + \theta x^{<1>} + b), \quad h^{<1>} = \theta_h a^{<1>} + b_h$$
$$\dots$$
$$a^{<N>} = a(\theta a^{<N-1>} + \theta x^{<N>} + b), \quad h^{<N>} = \theta_h a^{<N>} + b_h$$

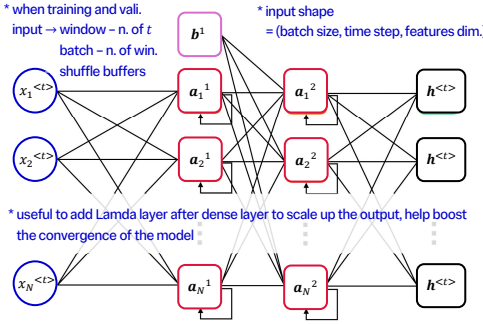
* same parameters and bias are reused every time step

p43 – p46

RNN recurrent neural network * vanilla (simple) RNN

consists of multiple recurrent cells, machine learns correlation within the features

* when training and vali. input → window – n. of t batch – n. of win. shuffle buffers
* input shape = (batch size, time step, features dim.)



* useful to add Lamda layer after dense layer to scale up the output, help boost the convergence of the model

number of parameters – in 1 RNN layer

$(\text{n. of input dimension} + 1) \times \text{n. cells} + \text{n. cells} \times \text{n. cells} = (N_\theta + N_b) + N_{\theta_h}$

* a typical case is prediction of the next step, $h_i^{<t>} = x_i^{<t+1>}$
* time is not a feature, it is an index to describe datapoints order

p47 – p57

loss function sum of the loss function of every timestep

$$J(\theta) = \sum_{i=0}^{N} J^{(i)}$$

* unlike training set in feedforward NN, which start from (1), time start from $< 0 >$

backpropagation in time

chain rule - * assume $a = 1$ for brevity

$$\frac{\partial J}{\partial a^{<t>}} = \frac{\partial J}{\partial a^{<N>}}\frac{\partial a^{<N>}}{\partial a^{<t>}} = \frac{\partial J}{\partial a^{<N>}}\frac{\partial a^{<N>}}{\partial a^{<N-1>}}\cdots\frac{\partial a^{<i+1>}}{\partial a^{<t>}} = \frac{\partial J}{\partial a^{<N>}}(\theta)^{N-1}$$

vanishing gradient - if θ is too small, $N - 1$ times θ will push next iteration θ become even smaller

* this can be solved
  1. use ReLU - unbounded $+ve$, gradient will not decay with large input
  2. use residual learning - skip connections, carry information through deep layers
  3. move to LSTMs, GRUs, reservoir computers

exploding gradient - if θ is too big, – next iteration θ become even bigger

* this can be solved
  1. clip the gradient – if $\theta > C$, then $\theta > \theta C/\|\theta\|$
  2. move to LSTMs, GRUs, reservoir computers

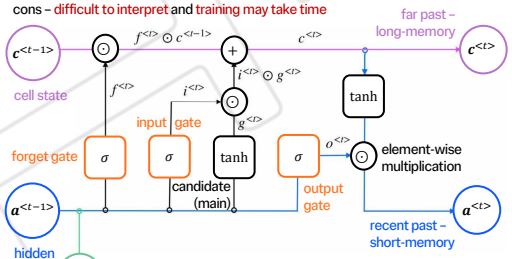
RNNs are not ideal for learning temporal correlations - have a short-term memory

p58 – p68

LSTMs long-short time memory units

pro – work very well in many applications

cons – difficult to interpret and training may take time



*n. of param. $= 4 \times$ simple RNN n. of param

forget state $f^{<t>} = \sigma(\theta_{a,f}a^{<t-1>} + \theta_f x^{<t>} + b_f) \approx 0$

input state $i^{<t>} = \sigma(\theta_{a,i}a^{<t-1>} + \theta_i x^{<t>} + b_i) \approx 1$

candidate state $g^{<t>} = tanh(\theta_{a,g}a^{<t-1>} + \theta_g x^{<t>} + b_g)$

output state $o^{<t>} = \sigma(\theta_{a,o}a^{<t-1>} + \theta_o x^{<t>} + b_o) \in (0, 1)$

cell state $c^{<t>} = f^{<t>} \odot c^{<t-1>} + i^{<t>} \odot g^{<t>}$

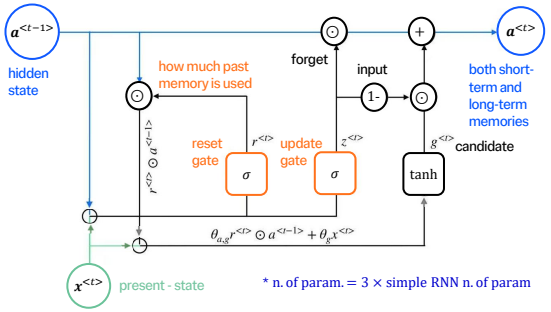
hidden state $a^{<t>} = o^{<t>} \odot \tanh(c^{<t>})$

p70 – p77

GRUs gated recurrent units

pro – work very well in many applications * e.g. time series forecasting , and complexity between RNNs and LSTMs * difficult to say whether GRUs or LSTMs perform better

cons – difficult to interpret, and training may take time * but less than LSTMs



* n. of param. $= 3 \times$ simple RNN n. of param

reset state $r^{<t>} = \sigma(\theta_{a,r}a^{<t-1>} + \theta_r x^{<t>} + b_r) \in (0, 1)$

update state $z^{<t>} = \sigma(\theta_{a,z}a^{<t-1>} + \theta_z x^{<t>} + b_z) \in (0, 1)$

* forget state $= z^{<t>}$ - how much old hidden state is kept

  input state $= 1 - z^{<t>}$ - how much new information is written

* if $z^{<t>} = 1$, forget is open and input is closed

candidate state $g^{<t>} = tanh(\theta_{a,g}r^{<t>} \odot a^{<t-1>} + \theta_g x^{<t>} + b_g)$

hidden state $a^{<t>} = z^{<t>} \odot a^{<t-1>} + (1 - z^{<t>}) \odot g^{<t>}$

p78 – p79

different tasks * $T_h = T_y$ in table below

Type	Input time step	Label time step	Example
one-to-one	$T_x = 1$	$T_y = 1$	traditional NN
one-to-many	$T_x = 1$	$T_y > 1$	music generation
many-to-one	$T_x > 1$	$T_y = 1$	sentiment 观点 classification
many-to-many	$T_x = T_y$		name entity recognition
	$T_x \neq T_y$		machine translation

7 **1.3**

clustering 聚类 group the instances based on their similarity without labels

customer segmentation 细分, data analysis, dimensionality reduction * known as vector quantization, useful in reduce order modelling, anomaly detection, image segmentation

centroid – k-means algorithm

seek continuous regions of densely packed instances – DBSCAN

hierarchical (seek clusters of clusters) – agglomerative 凝聚 and divisive 分裂 algorithms

1.4

k-Means clustering centroid-based heuristic 启发式 algorithm

* finds a solution may not optimal but in good trade-off between optimality and time

find centroid μ_j to minimize $\frac{1}{km}\sum_{j=1}^{k}\sum_{i=1}^{N_j}\|x^{(i)} - \mu_j\|^2$ = minimize moment of inertia

* N_j is number of points in cluster, j, N_j is unknown a priori, which is part of solution

1. choose a metric (distance function)

quantitatively defines the notion of similarity by measuring the distance between points in the feature space * choose Euclidean distance, $\|\cdot\|^2$, in this case

2. choose number of clusters k * significantly impact result, more clusters = less inertia

3. choose randomly the centroids μ_j , for $j = 1, \dots, k$ * run different initial., e.g. 10 times

4. compute the distances $\|x^{(i)} - \mu_j\|^2$, for $i = 1, \dots, m$ and $j = 1, \dots, k$

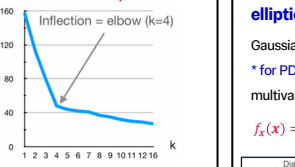
5. assign assign each instance $x^{(i)}$ to the closest centroid μ_j

6. compute the barycentre mean for each cluster $\mu_{j,\text{new}} = \frac{\sum_{i=1}^{m} z_j^{(i)} x^{(i)}}{\sum_{i=1}^{m} z_j^{(i)}}$, $z_j^{(i)} = 0$ or 1

7. update centroid overwrite centroid $\mu_j = \mu_{j,\text{new}}$ **8. compute the new distances**

9. repeat until $\mu_j - \mu_{j,\text{new}} \leq$ tol or reach maximum iterations * may only find local min.

elbow method find optimal k



limits

k-means perform pooly when clusters have varying sizes, densities, and non-spherical shapes * try kernel trick

k-mean is a 'hard' (non-probalistic) algorithm, all point is same important

* try soft (fuzzy) clustering or Gaussian (model) mixtures

1.5

DBSCAN density-based

* density-based spatial clustering of application with noise

pro – do not need to specify k, can find cluster with complex shapes, can classify noise, and only need to tune ε and N

cons – can misclassify, heavily depend on distance metric, perform poorly with big densities difference, and challenge to tune ε if estimation on data scales and distance are not accurate

1. choose a metric (distance function)

* choose Euclidean distance, $\|\cdot\|^2$

2. choose vicinity 邻近 ε

3. choose n. of point N point is need to classify a neighborhood as high density

4. select a new instance

5. count n. of instances around new instance satisfies $\|x^{(i)} - x^{(j)}\|^2 \leq \varepsilon$

if

n. of instances $+ 1$ (new ins.) $\geq N$ in ε − neighborhood, new instance is a core instance

* ε − neigh. may include other core ins.
* all neighboring core ins. form a cluster

else

not a core region, return to **4**

6. find anomalies classify as anomalies

* e.g. noise if all ins. that are not core ins. and do not have a core ins. in their neigh.

1.6

hierarchical clustering

heuristic and greedy algorithm

* greedy algorithm finds subproblem local optimum using heuristic algorithm, the solution of the problem at each stage is the input of next stage

bottom-up most common

1. choose a metric (distance function)

* choose Euclidean distance, $\|\cdot\|^2$

2. find distance $\|x^{(i)} - x^{(j)}\|^2$ for all i, j n. of dis. $= (m + 1 -$ current iter. $)^2$

3. merge closet pair in a new cluster

* criterion of merge is linkage criterion

choose centroid linkage clustering

* merged cluster midway of original ins.

4. repeat repeat **1.** and **2.** with $m - 1$ ins.

5. stop when have one large cluster

top-down known as divisive

1.7-1.8

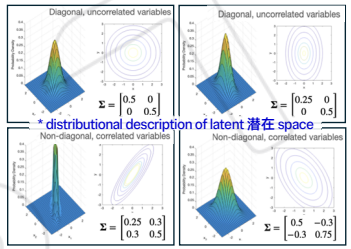
elliptic envelope for anomaly detection

Gaussian distribution – $f_x(x) = \frac{1}{\sqrt{2\pi\sigma^2}}\exp(-\frac{(x-\mu)^2}{2\sigma^2})$

* for PDF, $f_x(x) \sim \mathcal{N}(\mu, \sigma)$, μ is mean, is σ variance

multivariate Gaussian distribution –

$$f_x(x) = \frac{1}{\sqrt{(2\pi)^N \det(\boldsymbol{\Sigma})}}\exp\left(-\frac{\|x-\mu\|_\Sigma^2}{2}\right) \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$



* distributional description of latent 潜在 space

1. **assume** datapoint are Gaussian distributed
2. **compute** mean μ and covariance matrix Σ
3. **compute** Mahalanobis distance $D = \|x - \mu\|_\Sigma^2$ between test point (feature) and mean
4. **choose decision boundary** c * normal data region
5. **decide** if $D > c$, test point is an outlier - anomaly

* c=1 means around 68% datapoint in 1D distribution

8 **2**

PCA principal component analysis

principle variable - linear combination of original variables that captures the maximum variance

* in 2D correlated case, prin. var. is linear comb. of 2 var.

principle axes - direction of prin. var. in feature space

* all data input in prin. axes coordinates are uncorrelated

$$\max_{\mathbf{w}} \quad \text{Variance}_\mathbf{w} = \frac{1}{N-1}\sum_{i=1}^{N}\left[(x^{(i)} - \bar{x}) \cdot \mathbf{w}\right]^2$$
$$\text{subject to} \quad \|\mathbf{w}\|^2 = \mathrm{C} \text{ * usually set constant C = 1}$$

* $\bar{x} = \sum_{i=1}^{N} x^{(i)}/N$ is empirical mean
* normalize $\mathbf{w}$ to prevent its magnitude be variable

which has solution $\sum \hat{\mathbf{w}}_i = \sum(\mathbf{w}_i/\|\mathbf{w}_i\|) = \lambda_i \hat{\mathbf{w}}_i$

* $i = 1, \dots, \text{rank}(\boldsymbol{\Sigma})$, eigenvalue in decreasing order
* most and least important prin. axis are $\hat{\mathbf{w}}_1$ and $\hat{\mathbf{w}}_{\text{rank}}$
* var. are math independent (not connected by a fun.) less sensitive to input noise compare to linear regre.

pro – physical insight, and dimensionality reduction

cons – perform poorly with highly nonlinear data

***autoencoder** deconstruct – reconstruct

denoising, image reconstruction, colorization, ...

* encoder → bottleneck layer (latent space) → decoder

***autoencoder neural network**

loss – reconstruction loss in mse, mae, ...

***convolutional autoencoder**

model highly nonlinear, structured data (e.g. images) without assuming Gaussian distributions

* **transfer learning** re-train exist network

1. **freeze** N low-level layers, parameters will not update
2. **modify** output layers to have correct n. of output
   * can add preprocessing step to resize input as well
3. **re-train & re-validate** retrain output layers
4. **unfreeze** retrain more hidden layer if poor perform.
   * more training data, more unfreeze layers, reduce α

offline learning batch learning

online learning incremental learning

large α - quickly adapt to new data but forget old one